

Marjum 2026-07 — v0001 Geometry Snapshot Review

Milestone: captured, reviewable snapshot of the current fit state, built on `v0001_marjum_geometry` (marjum-2026-07 @ `ba52532`), per Aaron's review-gate on B2 (routed via experimental-strategist, 2026-09-13).

Question. Given the v0001 release and Aaron's three B2 rulings, what is the single reviewable statement of current transmitter/antenna position, uncertainty, and open — before any further optimization proceeds?

Aaron's rulings baked into this snapshot:

1. Cite the transmitter's ± 1.5 m bracket as the defensible value now (hard bracket independent-pair cross-checks, not a training residual).
2. **Log** the vertical-group disagreement as an open item; do not spend more optimization time on it now — it is smaller (~ 1 m / 94 m, $\sim 0.6^\circ$) than the $\sim 0.9^\circ$ Tx-direction uncertainty already carried to beam-analyst.
3. **Bracket the antenna position** like the transmitter, not a bare point estimate - unbracketed antenna position feeding a beam fit is the "not a result" pattern the warns against.

Constraint. This captures current state only. No new optimizer fits or further convergence attempts were run to produce it; the antenna bracket below is a deterministic envelope computation over already-computed fits (`marjum_antenna_uncertainty.py`), exactly mirroring how the transmitter's bracket was built (`marjum_tx_uncertainty.py`).

```
In [1]: import json
        from pathlib import Path

        import numpy as np
        import matplotlib.pyplot as plt

        WORKSPACE = Path('/home/aparsons/projects/eigsep/terrain')
        SNAP = WORKSPACE / 'v0001_marjum_geometry_snapshot'

        manifest = json.loads((SNAP / 'manifest.json').read_text())
        shared = json.loads((SNAP / 'shared.json').read_text())
        tx = json.loads((SNAP / 'transmitter_position.json').read_text())
        ant = json.loads((WORKSPACE / 'antenna_position_bracket.json').read_text())
        cameras = [json.loads(l) for l in (SNAP / 'cameras.jsonl').read_text().splitlines()]

        print('release:', manifest['release_id'], ' generated_utc:', manifest['generated_utc'])
        print('source commit (marjum-2026-07):', 'ba52532 (v0001_marjum_geometry_publish)')
        print('coverage:', manifest['coverage'])
```

```
release: v0001_marjum_geometry generated_utc: 2026-09-13T22:16:57Z
source commit (marjum-2026-07): ba52532 (v0001_marjum_geometry_publish)
coverage: {'excluded_camera_poses': 8, 'labeled_images': 37, 'usable_camera_poses': 29}
```

1. Inputs & provenance

The v0001 release is **not** on this repo's current checkout (`pointing-table-v0` in `marjum-2026-07`); it lives at `main @ ba52532` in that repository. Its `cameras.json`, `labels.json`, `manifest.json`, `shared.json`, and `transmitter_position.json` extracted read-only via `git show ba52532:marjum-2026-07/imgs/fits/v0001_marjum_geometry/<file>` into `v0001_marjum_geometry_snapshot/` along: notebook, and hashes below are checked against `manifest.json`'s own recorded v nothing here was regenerated.

The antenna bracket is new: `antenna_position_bracket.json`, produced by `marjum_antenna_uncertainty.py` in this repo, consuming only pre-existing `.npz`, `report.json` outputs (no new fits).

```
In [2]: print('=== v0001_marjum_geometry manifest ===')
print('status:', manifest['status'], 'supersedes:', manifest['supersedes'])
print('release artifacts (recorded sha256[:16]):')
for name, info in manifest['release_artifacts'].items():
    print(f"  {name:20s} {info['sha256'][:16]}")

print()
print('=== antenna_position_bracket.json provenance ===')
prov = ant['provenance']
print('generator:', prov['generator'], '@', prov['generator_commit'])
print('generated_utc:', prov['generated_utc'])
print('inputs:')
for i in prov['inputs']:
    print(f"  {i['path']:55s} {i['sha256'][:16]}")
```

```
=== v0001_marjum_geometry manifest ===
status: candidate  supersedes: None
release artifacts (recorded sha256[:16]):
  cameras.jsonl      2ed744f833f689e9
  labels.json        4657c42e494d3b56
  shared.json        234c1d5ab7142318
```

```
=== antenna_position_bracket.json provenance ===
generator: terrain/marjum_antenna_uncertainty.py @ 986e8cf
generated_utc: 2026-09-13T23:43:10Z
inputs:
  terrain/cv_position_absolute/baseline.npz      fd7163b974de99f3
  terrain/cv_position_absolute/orientations.npz  0b835d234498ac13
  terrain/cv_position_absolute/positions.npz     5ee3c01d2a7e4fdb
  terrain/cv_position_absolute/poses.npz         bdb00e55dbaf85fd
  terrain/cv_position_absolute/focal.npz         88271ca83a5833d6
  terrain/cv_antenna_repick_v1/fit_antenna.npz   f76d6ee58374e258
  terrain/cv_position_absolute/alternative.npz   227de81e10a53d2d
  terrain/cv_antenna_repick_v1/report.json       49998213041bf4e1
```

2. Coverage — cameras and labels

`manifest.json`: **37** labeled images, **29** usable camera poses, **8** excluded pose att
`cameras.jsonl` tags each row with `pose_status` (usable/excluded), `fit_proveni`

(established fixed camera vs. jointly fit against the transmitter ray), and `transmitter_conditioned` (True for the six cameras whose ray misses are circular evidence, per the transmitter caveats below).

```
In [3]: from collections import Counter

status_counts = Counter(c['pose_status'] for c in cameras)
prov_counts = Counter(c.get('fit_provenance') for c in cameras)
conditioned_counts = Counter(c.get('transmitter_conditioned') for c in cameras)
excluded = [c['image'] for c in cameras if c['pose_status'] != 'usable']

print('pose_status:', dict(status_counts))
print('fit_provenance:', dict(prov_counts))
print('transmitter_conditioned:', dict(conditioned_counts))
print('excluded images:', excluded)
```

```
pose_status: {'usable': 29, 'excluded': 8}
fit_provenance: {'joint terrain/features/transmitter fit': 6, 'established fixed camera': 23, None: 8}
transmitter_conditioned: {True: 6, False: 31}
excluded images: ['IMG_2219.HEIC', 'IMG_2225.HEIC', 'IMG_2227.HEIC',
'IMG_2228.HEIC', 'IMG_2230.HEIC', 'IMG_2241.HEIC', 'IMG_2242.HEIC',
'IMG_2243.HEIC']
```

3. Transmitter position — recommended value ar bracket

Unchanged from B2: `shared.json` / `transmitter_position.json` in this release. Aaron's ruling, **cite recommended_for_propagation**, not the `working_candidate` fails acceptance).

```
In [4]: rec = shared['transmitter']['recommended_for_propagation']
cand = shared['transmitter']['working_candidate']

print('recommended_for_propagation (CITE THIS):')
print('  position_enu_m:', rec['position_enu_m'])
print('  bound_m:', rec['bound_m'], '(', rec['bound_kind'], ')')
print('  stamp:', rec['stamp'])
print()
print('working_candidate (do not cite as final):')
print('  position_enu_m:', cand['position_enu_m'])
print('  accepted:', cand['accepted'], ' failing_checks:', cand['failing_
```

```
recommended_for_propagation (CITE THIS):
  position_enu_m: [1652.198, 2025.104, 1684.384]
  bound_m: 1.5 ( bounding radius, not a posterior sigma )
  stamp: marjum-2026-07/transmitter_position@v2+986e8cf
```

```
working_candidate (do not cite as final):
  position_enu_m: [1651.9749701444555, 2025.1048049047818, 1683.7038143160669]
  accepted: False failing_checks: ['2211_transmitter_reprojection']
```

3a. Vertical-group disagreement — logged as OPEN ITEM (r pursued, per ruling 2)

The independent-pair cross-check (cameras 2210/2211, the only ones never conditioned by the transmitter ray) places the transmitter **~1.0 m higher** than the working candidate, with only ~0.46 m horizontal offset. Aaron's ruling: this is a smaller effect (~1 m over ~9. antenna height, $\approx 0.6^\circ$) than the $\sim 0.9^\circ$ transmitter-direction uncertainty already carried downstream to beam-analyst — **log it, do not spend further optimizer time on**

```
In [5]: ip = tx['independent_pair_solution']
print('independent-pair (2210+2211) solution (ENU):', ip['solution_enu_m'])
print('mutual ray gap:', ip['mutual_ray_gap_m'], 'm')
print('offset from recommended value (dE, dN, dU, horiz, total):', ip['offset'])
print()
print('STATUS: OPEN ITEM, logged, not pursued further this milestone.')
print(ip['interpretation'])
```

independent-pair (2210+2211) solution (ENU): [1652.41, 2025.238, 1684.742]

mutual ray gap: 0.715 m

offset from recommended value (dE, dN, dU, horiz, total): {'dE': 0.435, 'dN': 0.133, 'dU': 1.038, 'horizontal': 0.455, 'total': 1.134}

STATUS: OPEN ITEM, logged, not pursued further this milestone.

The two unconditioned cameras agree with each other to 0.7 m and jointly place the transmitter ~1.0 m HIGHER than the v4 joint fit, with only ~0.45 m of horizontal offset. The disagreement is therefore almost purely vertical and is between the two camera GROUPS, not a single bad pick.

4. Antenna position — now bracketed (ruling 3)

Previously a bare point estimate with `uncertainty: null` in `shared.json` (the leave-one-out MCMC pilot was unconverged and supplied nothing usable). This notebook supplies a bracket built the same way as the transmitter's: envelope over independently-deferred generations, checked against the worst leave-one-out (LOO) held-out reprojection model. This is the closest thing this fit has to independent evidence, since each LOO refit never sees the view it is then scored against.

```
In [6]: u = ant['uncertainty']
print('best_estimate_enu_m:', ant['best_estimate_enu_m'])
print('bound_m:', u['bound_m'], '(', u['kind'], ')')
print('methodological_envelope_m:', u['basis']['methodological_envelope_m'])
print('leave_one_out_max_ray_miss_m:', u['basis']['leave_one_out_max_ray_miss_m'])
print('worst view:', u['basis']['leave_one_out_worst_view'])
print('rule:', u['basis']['rule'])
print()
print(u['interpretation'])
```

```

best_estimate_envelope_m: [1655.868, 2030.923, 1777.397]
bound_m: 1.7 ( bounding radius, not a posterior sigma )
methodological_envelope_m: 0.658
leave_one_out_max_ray_miss_m: 1.641  worst view: 2210
rule: bound = ceil(max(envelope, loo_max_ray_miss) * 10) / 10

```

Treat as a hard bracket on the antenna position, built the same way as the transmitter's bracket. The generation envelope (incremental scale-up sequence current repick) alone would give 0.658 m; the bound is instead set by the worst leave-one-out held-out reprojection miss (2210, 1.641 m), which is the closest thing this fit has to independent evidence – each L00 refit never sees the view is then scored against.

4a. Baseline diagnostics — generation envelope and L00 ray miss by view

```

In [7]: gens = ant['generations']
hdr = f"{'generation':32s} {'E':>9s} {'N':>9s} {'U':>9s} {'outlier':>8s}"
print(hdr); print('-' * len(hdr))
for name, g in gens.items():
    p = g['antenna_envelope_m']
    print(f"{'name':32s} {p[0]:9.3f} {p[1]:9.3f} {p[2]:9.3f} {str(g['outlier'])}")

print()
loo = ant['leave_one_out']
rm = loo['ray_miss_m']
hdr2 = f"{'image':8s} {'L00 ray_miss (m)':>17s} {'L00 position shift (m)'"
print(hdr2); print('-' * len(hdr2))
for k in sorted(rm, key=lambda k: -rm[k]):
    flag = ' <-- >1 m' if rm[k] > 1.0 else ''
    print(f"{'k':8s} {rm[k]:17.3f} {loo['position_shift_from_best_m'][k]:24s} {flag}")

```

generation	E	N	U	outlier
cv_position_absolute/baseline	1655.301	2030.311	1778.142	False
cv_position_absolute/orientations	1655.220	2030.611	1778.380	False
cv_position_absolute/positions	1655.357	2030.869	1778.536	False
cv_position_absolute/poses	1655.601	2030.915	1777.497	False
cv_position_absolute/focal	1655.646	2030.998	1777.452	False
cv_antenna_repick_v1	1655.868	2030.923	1777.397	False
cv_position_absolute/alternative	1643.589	2029.497	1781.677	True

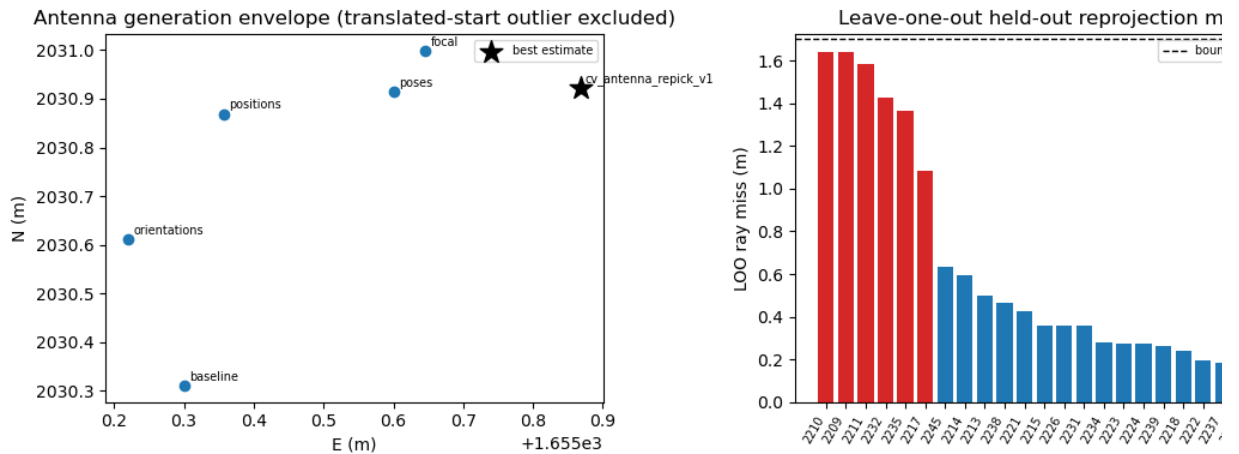
image	L00 ray_miss (m)	L00 position shift (m)
2210	1.641	0.037 <-- >1 m
2209	1.639	0.043 <-- >1 m
2211	1.584	0.038 <-- >1 m
2232	1.426	0.010 <-- >1 m
2235	1.364	0.020 <-- >1 m
2217	1.085	0.048 <-- >1 m
2245	0.634	0.042
2214	0.594	0.065
2213	0.498	0.044
2238	0.465	0.027
2221	0.426	0.034
2215	0.359	0.063
2226	0.358	0.042
2231	0.358	0.039
2234	0.281	0.015
2223	0.274	0.043
2224	0.273	0.060
2239	0.261	0.034
2218	0.241	0.074
2222	0.194	0.031
2237	0.183	0.158
2220	0.167	0.038
2216	0.106	0.008

```
In [8]: fig, ax = plt.subplots(1, 2, figsize=(11, 4))

names = [n for n in gens if not gens[n]['outlier_excluded_from_envelope']]
E = [gens[n]['antenna_enu_m'][0] for n in names]
N = [gens[n]['antenna_enu_m'][1] for n in names]
ax[0].scatter(E, N, c='tab:blue')
for n, e, nn in zip(names, E, N):
    ax[0].annotate(n.split('/')[1], (e, nn), fontsize=7, xytext=(3, 3),
be = ant['best_estimate_enu_m']
ax[0].scatter([be[0]], [be[1]], marker='*', s=200, c='k', label='best est
ax[0].set_xlabel('E (m)'); ax[0].set_ylabel('N (m)')
ax[0].set_title('Antenna generation envelope (translated-start outlier ex
ax[0].legend(fontsize=7)

keys = sorted(rm, key=lambda k: -rm[k])
vals = [rm[k] for k in keys]
x = np.arange(len(keys))
ax[1].bar(x, vals, color=['tab:red' if v > 1.0 else 'tab:blue' for v in v
ax[1].axhline(u['bound_m'], color='k', ls='--', lw=1, label=f"bound = {u[
ax[1].set_xticks(x); ax[1].set_xticklabels(keys, rotation=60, ha='right',
ax[1].set_ylabel('L00 ray miss (m)'); ax[1].set_title('Leave-one-out held
ax[1].legend(fontsize=7)
fig.tight_layout()
```

```
fig.savefig('_geom_snapshot_ant_diag.png', dpi=140)
plt.show()
```



4b. Antenna caveats

- 5 of 23 views (2209, 2210, 2211, 2217, 2235) show 1.0–1.6 m LOO ray misses and <0.16 m LOO position shift: removing any one of them barely moves the antenna estimate, but the estimate is a poor predictor of those views' labels. A local parameter-aware scan on 2209 (`ANTENNA_REPICK.md`) confirmed sub-meter camera adjustments do not remove this residual, and forcing it to zero raises held-out terrain residual ~13 px. **Logged as an open item, not pursued further** under this milestone new-fits constraint.
- The legacy MCMC pilot for this antenna is unconverged and contributes nothing to the bracket.
- LOO refits share camera poses/labels/distortion with the full fit; this is an internal robustness check, not a fully independent second instrument the way 2210/2211 share the transmitter.

5. Terrain map — camera positions, transmitter, antenna

Camera positions (all 37, usable vs. excluded) alongside the recommended transmitter position ($\pm$ bracket) and the newly bracketed antenna position.

```
In [9]: fig, ax = plt.subplots(figsize=(8, 7))
usable = [c for c in cameras if c['pose_status'] == 'usable']
excl = [c for c in cameras if c['pose_status'] != 'usable']

ux = [c['camera']['position_enu_m'][0] for c in usable]
uy = [c['camera']['position_enu_m'][1] for c in usable]
ax.scatter(ux, uy, c='tab:blue', s=25, label=f'usable cameras (n={len(usable)})')

excl_with_pose = [c for c in excl if c.get('camera')]
if excl_with_pose:
    ex = [c['camera']['position_enu_m'][0] for c in excl_with_pose]
    ey = [c['camera']['position_enu_m'][1] for c in excl_with_pose]
    ax.scatter(ex, ey, c='tab:gray', s=25, marker='x',
               label=f'excluded pose attempts with a pose (n={len(excl_with_pose)})')
```

```

# Excluded views without a solved pose (camera: null) can't be plotted; t
# were rejected before or without ever producing camera coordinates.
print(f"{len(excl) - len(excl_with_pose)} excluded views have no solved p
      f"(camera: null) and are omitted from the map.")

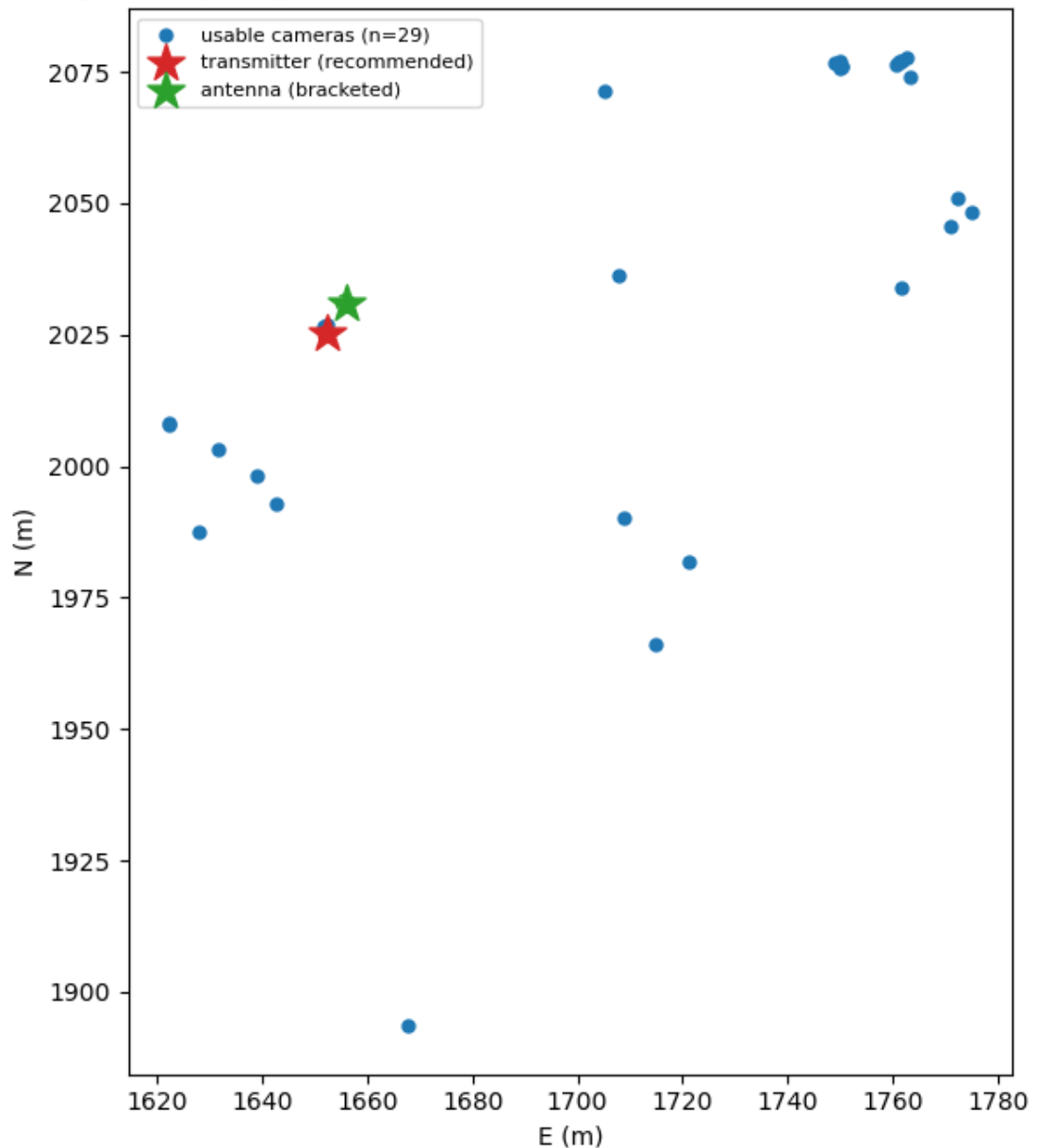
tx_e, tx_n, _ = rec['position_enu_m']
ant_e, ant_n, _ = ant['best_estimate_enu_m']
ax.scatter([tx_e], [tx_n], c='tab:red', marker='*', s=250, label='transmi
tx_circle = plt.Circle((tx_e, tx_n), rec['bound_m'], fill=False, color='t
ax.add_patch(tx_circle)
ax.scatter([ant_e], [ant_n], c='tab:green', marker='*', s=250, label='ant
ant_circle = plt.Circle((ant_e, ant_n), u['bound_m'], fill=False, color='
ax.add_patch(ant_circle)

ax.set_xlabel('E (m)'); ax.set_ylabel('N (m)')
ax.set_title('v0001 geometry snapshot: cameras, transmitter, antenna (bra
ax.set_aspect('equal')
ax.legend(fontsize=8, loc='upper left')
fig.tight_layout()
fig.savefig('_geom_snapshot_map.png', dpi=140)
plt.show()

```

8 excluded views have no solved pose (camera: null) and are omitted from the

v0001 geometry snapshot: cameras, transmitter, antenna (bracket radii to sc



6. Per-image horizon fit quality — ray-traced over

Reuses the Fit Inspector's own machinery (`HorizonImage` , `marjum_camera.rays` , `Terrain / working_grid` -anchored DEM ray tracing): ray-traced sky/ground bounds vs. the actual segmentation (green), at each image's pose, for **every camera behind snapshot**.

Which single file to load: the summary sections above deliberately have no single for the antenna (its bracket is an envelope over several generations, not one optimized). For this per-image visual check, however, one does exist: `cv_transmitter_joint_v1_fit_transmitter.npz` is the current working-candidate joint fit and already contains camera poses (23 established antenna cameras + 6 transmitter-conditioned) plus both antenna and transmitter positions used above — its `antenna` field is verified below to be byte-identical to `cv_antenna_repick_v1`'s (the antenna bracket's best estimate), and `marjum_transmitter_accept.py` already checks the 23 established camera poses for-byte against every prior generation. So this one file gives a self-consistent overlay

both targets; only the antenna's uncertainty bracket comes from elsewhere. The trar position it carries is the **working candidate**, not the accepted product — every pan below is titled accordingly.

Bug found and fixed while building this section: `marjum_bundle.working_grid` a related sign-handling fix in `Terrain.skyline`) were missing from the checked-out `marjum_bundle.py` , even though other scripts in this repo (`_render_horizon_overlays.py` , `marjum_horizons.py`) import and depend on it prior successful run's log (`_v4_vs_trial5_overlays.log`) shows it was present wh comparison was rendered. Restored verbatim from `/home/aparsons/projects/eigsep_old/terrain/marjum_bundle.py` (a known-good prior copy); its own subm: alignment self-check against `marjum_dem.npz` passed (`shift=(-1000.0, -1000.0` the `_sw` cache used below), and `test_marjum_bundle.py` still passes (5/7; the 2 f are pre-existing `ModuleNotFoundError`s in `eigsep_terrain` unrelated to this fix). Without this restore, every ray in this section would have been traced against terrair ~1.4 km off (1000 m in each of E and N), which would have made every overlay meaningless.

```
In [10]: import json as _json
import gc

import matplotlib.pyplot as plt
import numpy as np

try:
    import pillow_heif
    pillow_heif.register_heif_opener()
except Exception:
    pass

from eigsep_terrain.marjum_dem import MarjumDEM as _DEM
from eigsep_terrain.img import HorizonImage, PRM_ORDER
from marjum_bundle import working_grid
from marjum_camera import rays as camera_rays, project as camera_project

OVERLAY_FIT_FILE = 'cv_transmitter_joint_v4/fit_transmitter.npz'

dem = working_grid(_DEM(cache_file='marjum_dem_sw.npz'))
print('DEM working-grid shift:', dem._working_shift)
meta_all = _json.loads((WORKSPACE / 'meta.json').read_text())

with np.load(OVERLAY_FIT_FILE) as fit_npz:
    ov_keys = [str(k) for k in fit_npz['keys']]
    ov_cameras = np.asarray(fit_npz['cameras'], float)
    ov_distortion = np.asarray(fit_npz['distortion'], float)
    ov_antenna = np.asarray(fit_npz['antenna'], float)
    ov_transmitter = np.asarray(fit_npz['transmitter'], float)
    ov_shapes = np.asarray(fit_npz['shapes'], int)
    ov_conditioned = set(str(k) for k in fit_npz['transmitter_conditioned

camera_by_key = dict(zip(ov_keys, ov_cameras))
distortion_by_key = dict(zip(ov_keys, ov_distortion))
shapes_by_key = dict(zip(ov_keys, ov_shapes))

# Cross-check: this file's antenna must match the bracket's best estimate
```

```

# and its established (non-conditioned) camera poses must match
# cv_antenna_repick_v1 byte-for-byte, or this section's premise (one file
# two consistent targets) is wrong.
assert np.allclose(ov_antenna, ant['best_estimate_enu_m'], atol=1e-6), 'a
with np.load(WORKSPACE / 'cv_antenna_repick_v1' / 'fit_antenna.npz') as f
    a_keys = [str(k) for k in f2['keys']]
    a_cams = dict(zip(a_keys, f2['cameras']))
max_established_diff = max(
    np.abs(camera_by_key[k] - a_cams[k]).max() for k in a_keys if k in ca
)
print(f'OVERLAY_FIT_FILE = {OVERLAY_FIT_FILE!r}: {len(ov_keys)} cameras '
      f'({len(ov_conditioned)} transmitter-conditioned, {len(ov_keys) - 1
print('antenna field matches bracket best estimate:', True)
print('max established-camera param diff vs cv_antenna_repick_v1:', max_e
print('working candidate transmitter position:', ov_transmitter, '(NOT th

```

```

DEM working-grid shift: (-1000.0, -1000.0)
OVERLAY_FIT_FILE = 'cv_transmitter_joint_v4/fit_transmitter.npz': 29 cameras
transmitter-conditioned, 23 established)
antenna field matches bracket best estimate: True
max established-camera param diff vs cv_antenna_repick_v1: 0.0
working candidate transmitter position: [1651.97497014 2025.1048049 1683.703
(NOT the accepted product)

```

```

In [11]: def distorted_image_rays(img, key, decimate):
    rr, cc = np.mgrid[0:img.npix_y:decimate, 0:img.npix_x:decimate]
    shape = (img.npix_y, img.npix_x)
    world = camera_rays(camera_by_key[key], shape, np.c_[cc.ravel(), rr.r
                    distortion_by_key[key])
    return world.T.reshape(3, *rr.shape)

def skyline_error_px(actual_sky, model_sky, decimate):
    # Per-column skyline position error, in full-resolution pixels. For e
    # column the skyline is the lowest sky pixel (masks are bottom-up, so
    # first True scanning up from row 0); columns that are entirely sky o
    # entirely ground carry no skyline and are dropped.
    def rows(mask):
        valid = mask.any(0) & ~mask.all(0)
        return np.where(valid, np.argmax(mask, axis=0), np.nan)
    a, m = rows(actual_sky), rows(model_sky)
    ok = ~np.isnan(a) & ~np.isnan(m)
    if not ok.any():
        return np.nan, np.nan
    err = np.abs(a[ok] - m[ok]) * decimate
    return float(np.median(err)), float(np.percentile(err, 90))

def sky_mismatch_frac(actual_sky, model_sky):
    # Fraction of pixels where the ray-traced sky/ground call disagrees w
    # the actual segmentation. Note: HorizonImage._raster_boundary, which
    # original Fit Inspector's SSD metric used, is not present in the
    # currently installed eigsep_terrain (another environment-drift gap,
    # alongside the marjum_bundle.working_grid one above) -- this simpler
    # whole-mask disagreement fraction is used instead; it is monotonic i
    # the same sense (lower = better fit) but is not numerically comparab
    # to any SSD value quoted elsewhere in this repo.
    return float(np.mean(actual_sky != model_sky))

```

```

ncols = 5
nrows = int(np.ceil(len(ov_keys) / ncols))
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(3.2 * ncols,
axes = np.atleast_1d(axes).ravel())

mismatch = {}
skyline_med = {}
skyline_p90 = {}
ant_rows = []
transmitter_rows = []
ant_pixel_rays = {}
transmitter_pixel_rays = {}
ordered_keys = sorted(ov_keys, key=lambda k: (k not in ov_conditioned, k))
for ax, key in zip(axes, ordered_keys):
    img = HorizonImage(f'marjum-2026-07/IMG_{key}.HEIC', meta=meta_all, p
    img.set_prms([camera_by_key[key][i] for i in range(len(PRM_ORDER))])

    if hasattr(img, 'meta') and 'ant_px' in img.meta:
        ax_, ay_ = img.meta['ant_px']
        shape = (img.npix_y, img.npix_x)
        predicted, _ = camera_project(camera_by_key[key], shape, ov_anten
        col_pred, row_pred = predicted[0]
        ant_rows.append((key, ax_, ay_, col_pred, row_pred, np.hypot(col_
        # World-space ray through the manually picked pixel, for the terr
        ant_pixel_rays[key] = camera_rays(camera_by_key[key], shape, [[ax

    if hasattr(img, 'meta') and 'transmitter_px' in img.meta:
        tx_, ty_ = img.meta['transmitter_px']
        shape = (img.npix_y, img.npix_x)
        predicted, _ = camera_project(camera_by_key[key], shape, ov_trans
        col_pred, row_pred = predicted[0]
        transmitter_rows.append((key, tx_, ty_, col_pred, row_pred, np.hy
        transmitter_pixel_rays[key] = camera_rays(camera_by_key[key], sha

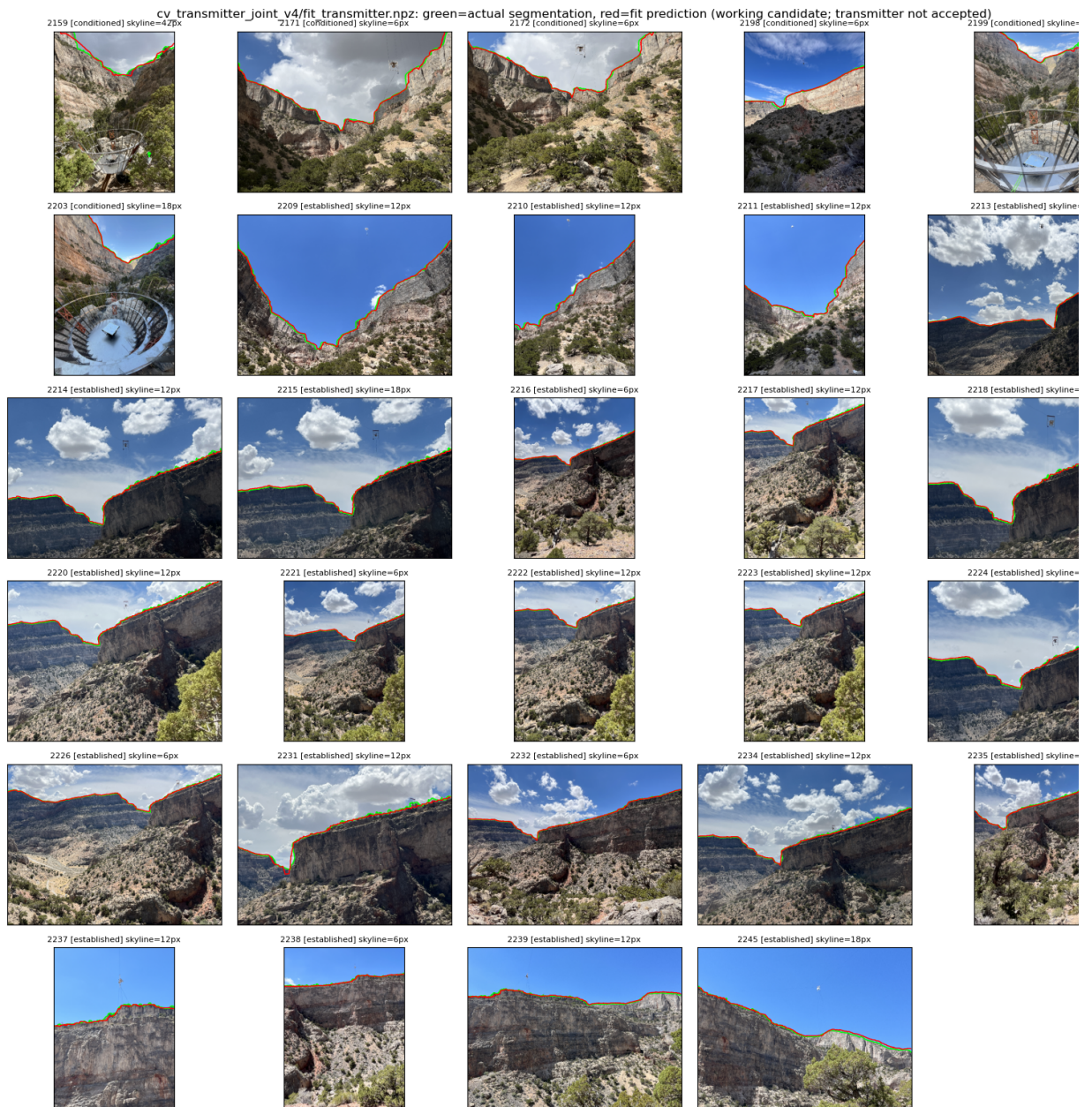
    sl = slice(None, None, 6)
    actual_sky = img.sky_mask[sl, sl]
    r = img.ray_distance(dem, distorted_image_rays(img, key, 6))
    model_sky = np.isnan(r)
    mismatch[key] = sky_mismatch_frac(actual_sky, model_sky)
    skyline_med[key], skyline_p90[key] = skyline_error_px(actual_sky, mod
    ax.imshow(img.img[sl, sl], origin='lower')
    ax.contour(actual_sky.astype(float), levels=[0.5], colors='lime', lin
    ax.contour(model_sky.astype(float), levels=[0.5], colors='red', linew
    tag = 'conditioned' if key in ov_conditioned else 'established'
    ax.set_title(f'{key} [{tag}] skyline={skyline_med[key]:.0f}px', fonts
    ax.set_xticks([]); ax.set_yticks([])

    del img
    gc.collect()

for ax in axes[len(ov_keys):]:
    ax.axis('off')

fig.suptitle(f'{OVERLAY_FIT_FILE}: green=actual segmentation, red=fit pre
            '(working candidate; transmitter not accepted)')
plt.tight_layout()
fig.savefig('_geom_snapshot_horizon_overlays.png', dpi=130)
plt.show()

```



6a. Horizon fit quality summary and reprojection residuals

```
In [12]: fig, axs = plt.subplots(2, 1, figsize=(10, 8), sharex=True)
order = sorted(ov_keys, key=lambda k: skyline_med[k])
colors = ['tab:orange' if k in ov_conditioned else 'tab:blue' for k in order]

axs[0].bar(range(len(order)), [skyline_med[k] for k in order], color=colors)
axs[0].bar(range(len(order)), [skyline_p90[k] - skyline_med[k] for k in order],
           bottom=[skyline_med[k] for k in order], color=colors, alpha=0.5)
axs[0].set_ylabel('skyline error (px)\nsolid = median, faded = to 90th percentile')
axs[0].set_title(f'{OVERLAY_FIT_FILE}: per-image horizon fit quality '
                '(orange = transmitter-conditioned)')

axs[1].bar(range(len(order)), [100 * mismatch[k] for k in order], color=colors)
axs[1].set_ylabel('sky/ground mask mismatch (%)')
axs[1].set_xticks(range(len(order)))
axs[1].set_xticklabels(order, rotation=90, fontsize=8)
plt.tight_layout()
fig.savefig('_geom_snapshot_mismatch.png', dpi=130)
plt.show()
```

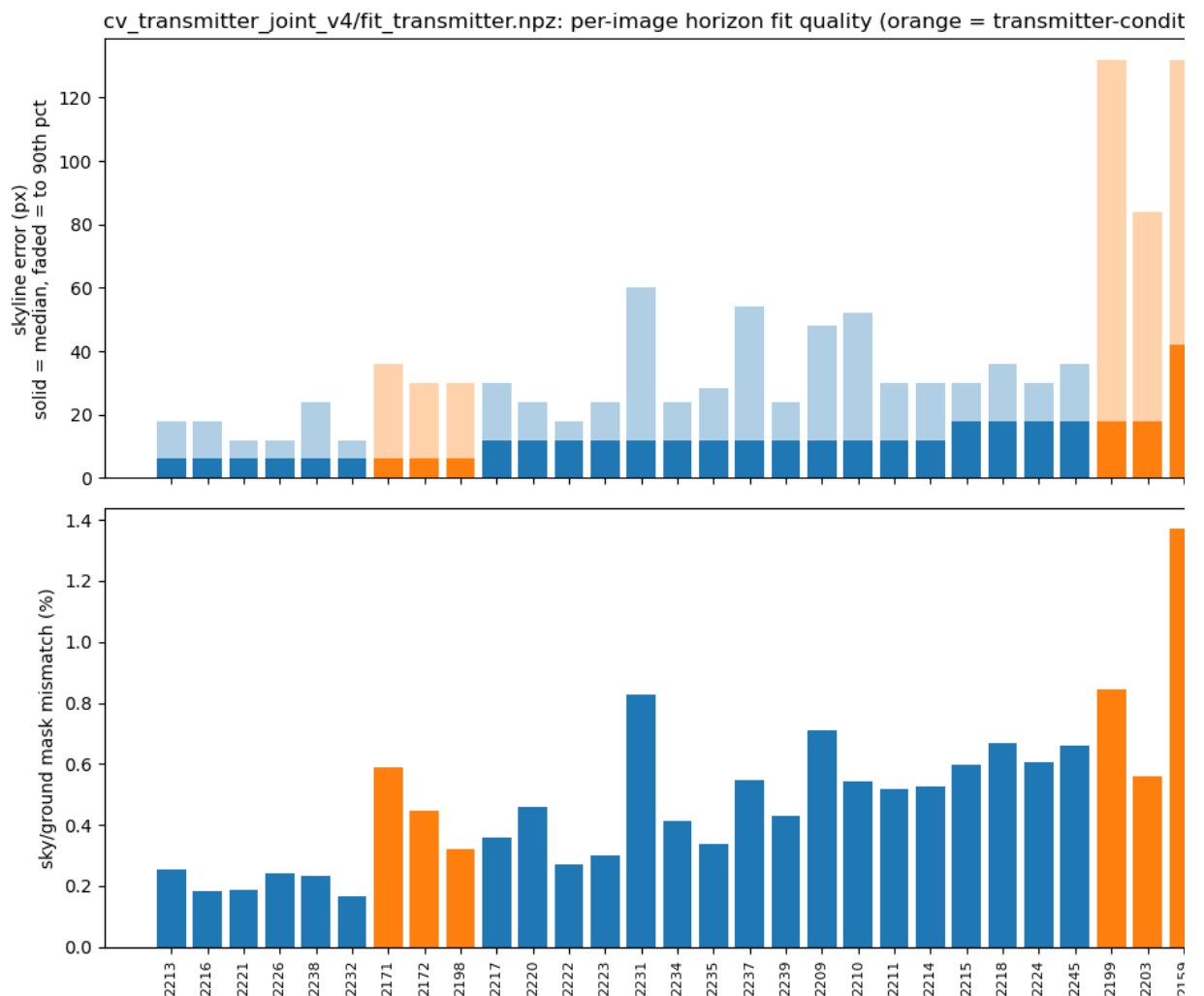
```

worst = max(ov_keys, key=lambda k: skyline_med[k])
print(f'Worst skyline error: {worst} at {skyline_med[worst]:.0f} px media
      f'({skyline_p90[worst]:.0f} px at the 90th pct); '
      f'2203 for comparison: {skyline_med["2203"]:.0f} px median.')
print(f'Note the two panels disagree about which images are bad: the mask
      f'metric (bottom) is dominated by the large, trivially-correct sky
      f'areas and is nearly blind to a displaced skyline. The skyline err
      f'the diagnostic one. Aaron flagged 2159 by eye from the overlays b
      f'metric here did -- the mask metric rated it 1.4%, which reads as

print()
print('=== Antenna-pixel reprojection (established cameras only carry ant
hdr = f'{"image":8s} {"picked":>18s} {"predicted":>18s} {"resid_px":>9s}"
print(hdr); print('-' * len(hdr))
for key, ax_, ay_, cp, rp, resid in sorted(ant_rows, key=lambda r: -r[-1])
    print(f"{key:8s} ({ax_:7.1f},{ay_:7.1f}) ({cp:7.1f},{rp:7.1f}) {resid

print()
print('=== Transmitter-pixel reprojection (all 8 transmitter-labeled came
print(hdr); print('-' * len(hdr))
for key, tx_, ty_, cp, rp, resid in sorted(transmitter_rows, key=lambda r
    tag = ' (independent check)' if key not in ov_conditioned else ' (con
    print(f"{key:8s} ({tx_:7.1f},{ty_:7.1f}) ({cp:7.1f},{rp:7.1f}) {resid

```



Worst skyline error: 2159 at 42 px median (132 px at the 90th pct); 2203 for comparison: 18 px median.

Note the two panels disagree about which images are bad: the mask-mismatch me (bottom) is dominated by the large, trivially-correct sky and ground areas an nearly blind to a displaced skyline. The skyline error (top) is the diagnosti Aaron flagged 2159 by eye from the overlays before either metric here did -- mask metric rated it 1.4%, which reads as fine.

=== Antenna-pixel reprojection (established cameras only carry ant_px) ===

image	picked		predicted		resid_px
2210	(1364.9, 3885.1)	(1314.1, 3884.5)			50.78
2211	(1212.5, 3724.0)	(1168.4, 3715.2)			45.02
2217	(1585.3, 3865.8)	(1554.4, 3834.9)			43.65
2209	(2434.2, 2746.5)	(2393.7, 2753.9)			41.16
2235	(1933.5, 3388.6)	(1901.0, 3364.7)			40.35
2231	(3299.2, 2272.4)	(3278.0, 2275.6)			21.44
2232	(1914.0, 1907.5)	(1892.8, 1905.4)			21.29
2245	(2273.5, 2236.4)	(2258.8, 2250.0)			20.08
2215	(2606.6, 2312.6)	(2615.2, 2329.6)			19.05
2213	(2145.2, 2788.0)	(2126.6, 2786.0)			18.68
2238	(1785.0, 3530.0)	(1800.4, 3531.4)			15.47
2214	(2227.6, 2118.8)	(2240.1, 2125.6)			14.21
2218	(2318.4, 2558.4)	(2304.9, 2557.5)			13.57
2221	(2129.5, 2995.2)	(2138.5, 3004.3)			12.83
2226	(3205.9, 2659.6)	(3197.5, 2650.0)			12.75
2224	(2401.8, 1872.8)	(2410.6, 1881.8)			12.65
2223	(1909.3, 3554.7)	(1903.8, 3545.6)			10.66
2239	(1122.2, 2464.0)	(1131.3, 2464.8)			9.16
2234	(2557.2, 1815.5)	(2564.5, 1811.1)			8.48
2222	(2082.8, 3503.5)	(2075.8, 3503.9)			7.03
2220	(2232.7, 2554.1)	(2229.9, 2548.2)			6.59
2237	(1663.1, 3169.2)	(1666.9, 3165.7)			5.17
2216	(1610.3, 2949.4)	(1612.6, 2948.5)			2.46

=== Transmitter-pixel reprojection (all 8 transmitter-labeled cameras) ===

image	picked		predicted		resid_px

2211	(1590.3,	679.9)	(1579.6,	546.7)	133.64 (independent check)
2210	(2029.7,	710.4)	(2030.5,	647.4)	63.03 (independent check)
2203	(1443.3,	957.2)	(1443.0,	957.4)	0.32 (conditioned, circular
2171	(3486.8,	264.0)	(3486.8,	264.2)	0.19 (conditioned, circular
2199	(1521.8,	518.6)	(1521.9,	518.5)	0.14 (conditioned, circular
2172	(2117.5,	602.8)	(2117.5,	602.6)	0.12 (conditioned, circular
2198	(1431.7,	1162.3)	(1431.6,	1162.3)	0.05 (conditioned, circular
2159	(1482.5,	780.4)	(1482.5,	780.4)	0.01 (conditioned, circular

6b. Terrain map: camera positions, headings, and label ray

Restores the Fit Inspector's own terrain-map cell: hillshade-relief DEM background (`terrain_hillshade_cmap.plot_terrain_hillshade`), every camera's position ar heading arrow, plus the world-space ray traced through each image's manually pick label (not through the fitted antenna/ transmitter position) — dashed for antenna-pix dotted for transmitter-pixel rays. Rays that converge tightly on the starred antenna/ transmitter markers corroborate the fit; any that miss by a wide, consistent margin f pose or label problem (see §6d below for 2159).

```

In [13]: from marjum_bundle import WORKING_GRID_SHIFT_M
from terrain_hillshade_cmap import plot_terrain_hillshade
from matplotlib.patches import Circle

se, sn = dem._working_shift # (-1000., -1000.) for the _sw cache

# dem's get_en/interp_alt are working_grid-patched to no-arg calls only,
# get_tile (which calls get_en(erng_m, ...) with arguments) cannot be cal
# on `dem` directly. Use a second, unpatched DEM handle for the tile only
# in the cache's own raw coordinate frame, then shift the returned E/N ba
# into working-grid metres to match ov_antenna/ov_transmitter/camera_by_k
dem_raw = _DEM(cache_file='marjum_dem_sw.npz')
e0, n0 = ov_antenna[0], ov_antenna[1]
rng = 200
E_raw, N_raw, U_map = dem_raw.get_tile(erng_m=(e0 - se - rng, e0 - se + r
nrng_m=(n0 - sn - rng, n0 - sn +
E_map, N_map = E_raw + se, N_raw + sn

fig, ax = plt.subplots(figsize=(10, 9))
terrain_im = plot_terrain_hillshade(ax, U_map, res=float(dem.res),
extent=(E_map[0], E_map[-1], N_map[0]
fig.colorbar(terrain_im, ax=ax, label='Elevation [m]')
ax.set_xlabel('East [m]'); ax.set_ylabel('North [m]')

cam_colors = plt.cm.tab20(np.linspace(0, 1, len(ov_keys)))
arrow_len = 5
EXTRA_LEN = 30
pixel_ray_label = 'antenna-pixel ray'
transmitter_ray_label = 'transmitter-pixel ray'

def ray_endpoint(e, n, direction, maxlen=180):
    ray_e, ray_n = direction[:2]
    xlim, ylim = ax.get_xlim(), ax.get_ylim()
    edge_t = []
    if ray_e > 0:
        edge_t.append((xlim[1] - e) / ray_e)
    elif ray_e < 0:
        edge_t.append((xlim[0] - e) / ray_e)
    if ray_n > 0:
        edge_t.append((ylim[1] - n) / ray_n)
    elif ray_n < 0:
        edge_t.append((ylim[0] - n) / ray_n)
    edge_t = [t for t in edge_t if t > 0]
    t = min(min(edge_t), maxlen) if edge_t else maxlen
    return e + t * ray_e, n + t * ray_n

for key, color in zip(ov_keys, cam_colors):
    e, n, ph = camera_by_key[key][0], camera_by_key[key][1], camera_by_ke
    loc = np.array([e, n])
    d_ant = np.linalg.norm(loc - ov_antenna[:2])
    d_tx = np.linalg.norm(loc - ov_transmitter[:2])
    ax.plot(e, n, 'o', color=color, ms=6, mec='k', mew=0.5)
    de, dn = arrow_len * np.cos(ph), arrow_len * np.sin(ph)
    ax.plot([e, e + de], [n, n + dn], '--', color=color, lw=1.5)
    if key in ant_pixel_rays:
        endpoint = ray_endpoint(e, n, ant_pixel_rays[key], maxlen=d_ant +
ax.plot([e, endpoint[0]], [n, endpoint[1]], '--', color=color, lw

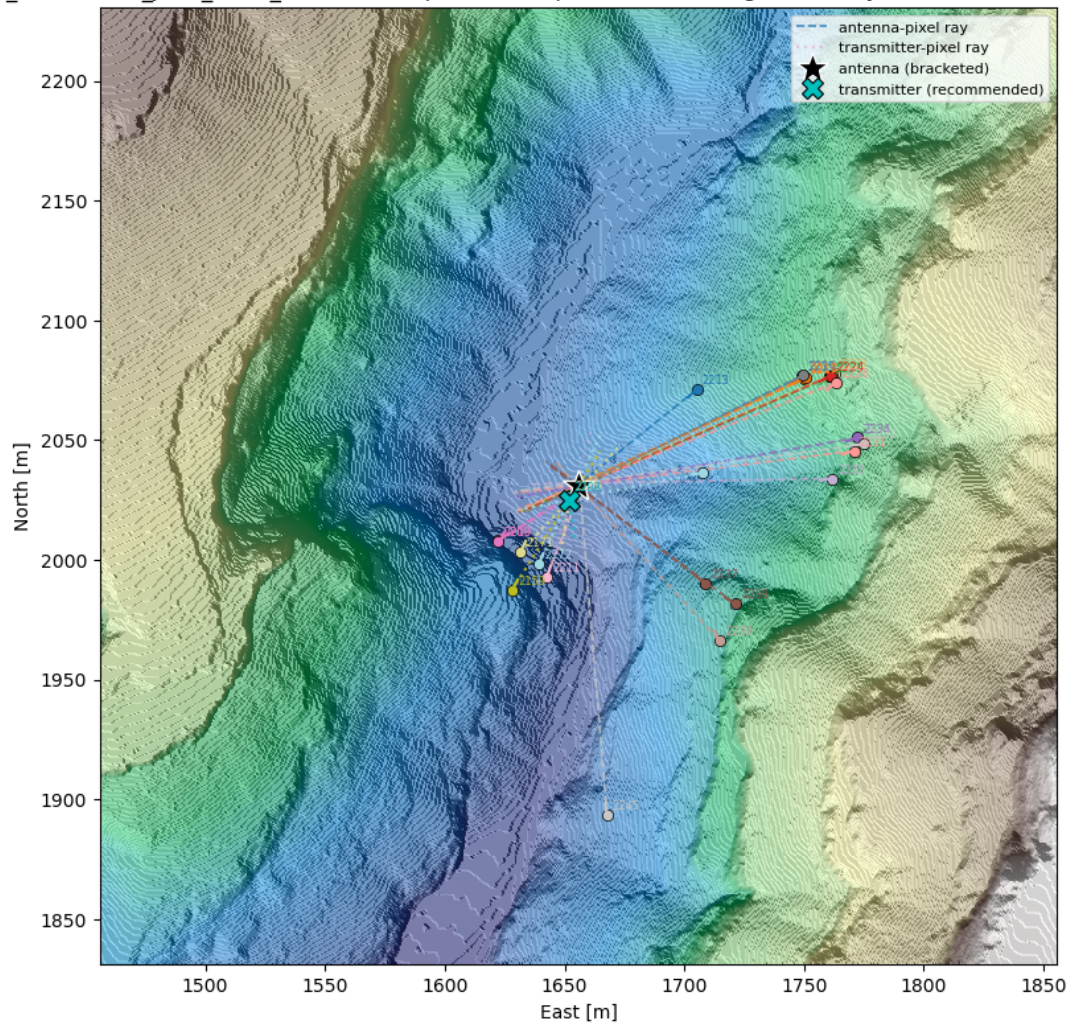
```

```

        label=pixel_ray_label)
    pixel_ray_label = None
    if key in transmitter_pixel_rays:
        endpoint = ray_endpoint(e, n, transmitter_pixel_rays[key], maxlen
        ax.plot([e, endpoint[0]], [n, endpoint[1]], ':', color=color, lw=
            alpha=0.9 if key in ov_conditioned else 0.35, label=trans
            transmitter_ray_label = None
        ax.annotate(key, (e, n), fontsize=6, color=color, xytext=(3, 3), text
    ax.plot(ov_antenna[0], ov_antenna[1], 'k*', ms=18, mec='w', mew=1, label=
    ax.add_patch(Circle((ov_antenna[0], ov_antenna[1]), u['bound_m'], fill=Fa
    ax.plot(rec['position_enu_m'][0], rec['position_enu_m'][1], 'cX', ms=12,
        label='transmitter (recommended)')
    ax.add_patch(Circle((rec['position_enu_m'][0], rec['position_enu_m'][1]),
        fill=False, color='c', ls='--'))
    ax.legend(loc='best', fontsize=8)
    ax.set_title(f'{OVERLAY_FIT_FILE}: camera positions, headings, label rays
    plt.tight_layout()
    fig.savefig('_geom_snapshot_hillshade_map.png', dpi=140)
    plt.show()

```

cv_transmitter_joint_v4/fit_transmitter.npz: camera positions, headings, label rays, antenna and transr



6c. Antenna and transmitter position comparison, zoomed

For every image with a pick: the hand-labeled pixel (magenta `+`) against the fit's pc projected into that image (cyan `x`). Restores the Fit Inspector's zoomed-comparison (antenna: cropped inline; transmitter: via

`marjum_inspector.plot_landmark_comparison`, reused as-is).

```
In [14]: ncols = 5
nrows = int(np.ceil(len(ant_rows) / ncols)) or 1
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(3.4 * ncols,
axes = np.atleast_1d(axes).ravel()

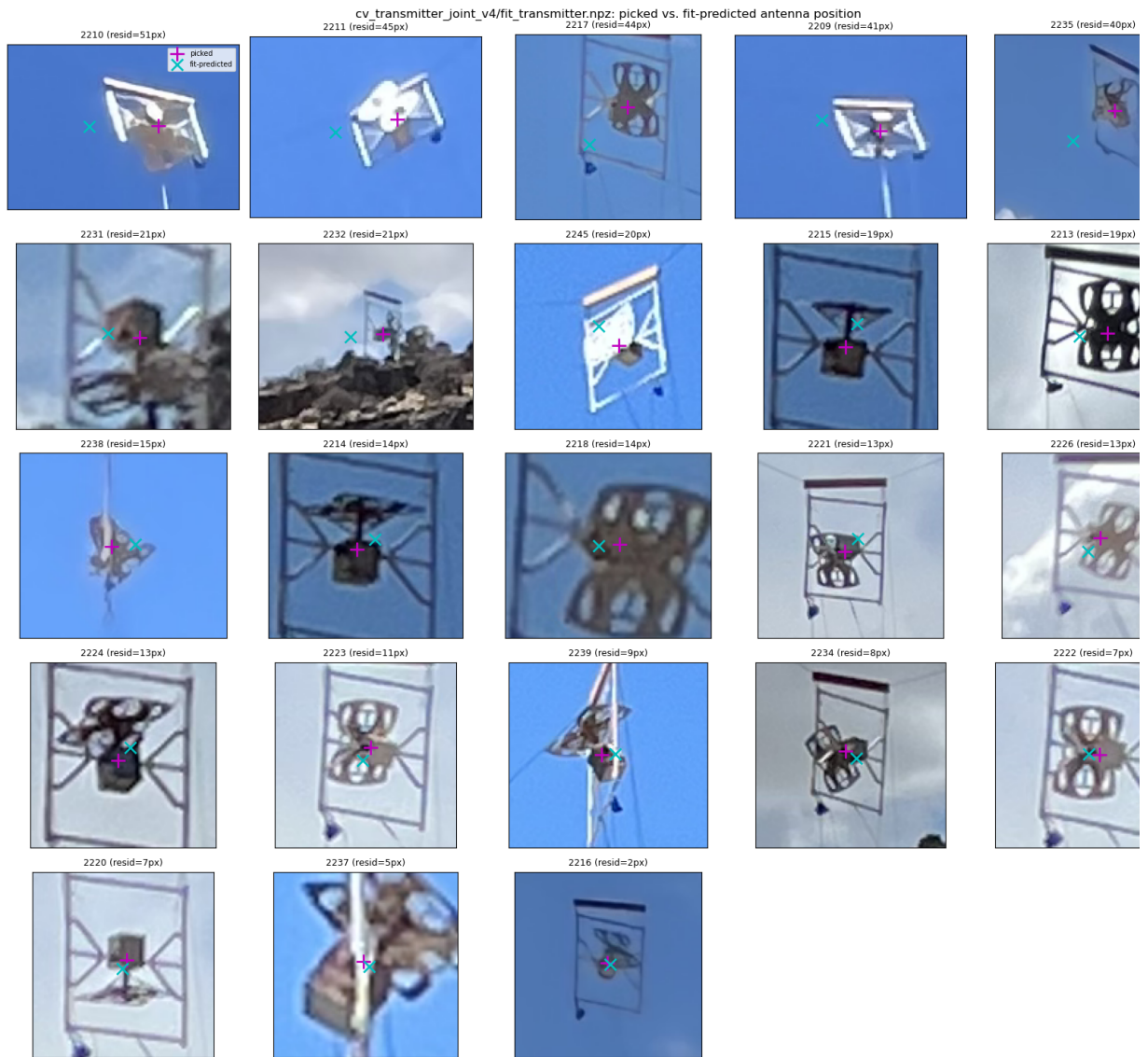
pad = 60
for ax, (key, ax_, ay_, cp, rp, resid) in zip(axes, sorted(ant_rows, key=
img = HorizonImage(f'marjum-2026-07/IMG_{key}.HEIC', meta=meta_all, p
img.set_prms([camera_by_key[key][i] for i in range(len(PRM_ORDER))])

    xmin, xmax = sorted((ax_, cp))
    ymin, ymax = sorted((ay_, rp))
    x0, x1 = int(max(xmin - pad, 0)), int(min(xmax + pad, img.npix_x))
    y0, y1 = int(max(ymin - pad, 0)), int(min(ymax + pad, img.npix_y))
    crop = img.img[y0:y1, x0:x1]

    ax.imshow(crop, origin='lower', extent=(x0, x1, y0, y1))
    ax.plot(ax_, ay_, 'm+', ms=14, mew=2, label='picked')
    ax.plot(cp, rp, 'cx', ms=12, mew=2, label='fit-predicted')
    ax.set_xlim(x0, x1); ax.set_ylim(y0, y1)
    ax.set_title(f'{key} (resid={resid:.0f}px)', fontsize=9)
    ax.set_xticks([]); ax.set_yticks([])

    del img, crop
    gc.collect()

for ax in axes[len(ant_rows):]:
    ax.axis('off')
if ant_rows:
    axes[0].legend(loc='upper right', fontsize=7)
fig.suptitle(f'{OVERLAY_FIT_FILE}: picked vs. fit-predicted antenna posit
plt.tight_layout()
fig.savefig('_geom_snapshot_ant_zoom.png', dpi=130)
plt.show()
```



```
In [15]: # marjum_inspector.plot_landmark_comparison imports eigsep_terrain.imagei
# which -- like eigsep_terrain.tiepoints/fitio (test_marjum_bundle.py) an
# HorizonImage._raster_boundary (SS6) -- is not present in the currently
# installed eigsep_terrain (another instance of the same environment-drif
# family). Reimplemented inline using the same plain-matplotlib image loa
# HorizonImage itself already uses successfully elsewhere in this noteboo
# rather than patching the installed package.
def load_image_flipped(path):
    return np.flipud(plt.imread(path))

def plot_landmark_comparison(rows, fit_file, landmark='transmitter', ncol
rows = sorted(rows, key=lambda r: -r[-1])
fig, axes = plt.subplots(max(1, int(np.ceil(len(rows) / ncols))), nco
figsize=(3.4 * ncols, 3.0 * max(1, int(np.ce
axes = np.atleast_1d(axes).ravel()
for ax, (key, x, y, px, py, resid) in zip(axes, rows):
    image = load_image_flipped(f'marjum-2026-07/IMG_{key}.HEIC')
    h, w = image.shape[:2]
    x0 = int(np.clip(min(x, px) - pad, 0, w - 1)); x1 = int(np.clip(m
    y0 = int(np.clip(min(y, py) - pad, 0, h - 1)); y1 = int(np.clip(m
    crop = image[y0:y1, x0:x1].copy(); del image
    ax.imshow(crop, origin='lower', extent=(x0, x1, y0, y1))
    ax.plot(x, y, 'm+', ms=14, mew=2, label='picked')
    ax.plot(px, py, 'cx', ms=12, mew=2, label='fit-predicted')
```

```

        ax.set(xlim=(x0, x1), ylim=(y0, y1), xticks=[], yticks=[], title=
            if not (0 <= px < w and 0 <= py < h):
                ax.text(.02, .02, 'Prediction outside image', transform=ax.tr
    for ax in axes[len(rows):]:
        ax.axis('off')
    if rows:
        axes[0].legend(loc='upper right', fontsize=7)
    fig.suptitle(f'{fit_file}: picked vs. fit-predicted {landmark} position')
    fig.tight_layout()
    return fig, axes

transmitter_zoom_fig, transmitter_zoom_axes = plot_landmark_comparison(
    transmitter_rows, OVERLAY_FIT_FILE, landmark='transmitter')
transmitter_zoom_fig.savefig('_geom_snapshot_tx_zoom.png', dpi=130)

```

cv_transmitter_joint_v4/fit_transmitter.npz: picked vs. fit-predicted transmitter position



6d. Investigation: 2159 vs. 2203

Aaron's finding: the horizon overlay for 2159 is a poor match, and 2159's position heading should be very similar to 2203's — he later refined the separation to probably a couple of metres, against the ~13 m my first pass read off the EXIF GPS.

Both numbers are reported below and they do not actually conflict. Each photo's EXIF carries the phone's own stated horizontal positioning error: **±11.8 m for 2159** and **±4.7 m for 2203** (2159's fix is poor, as expected in a canyon with restricted sky view; separation therefore carries a combined 1σ of ~12.7 m, so the 13.2 m EXIF reading is a **1.0 σ** measurement — statistically indistinguishable from Aaron's couple of metres, close to zero. My previous pass presented that 13.2 m as if it were a precise confirmation; it isn't, and the position comparison on its own cannot settle anything.

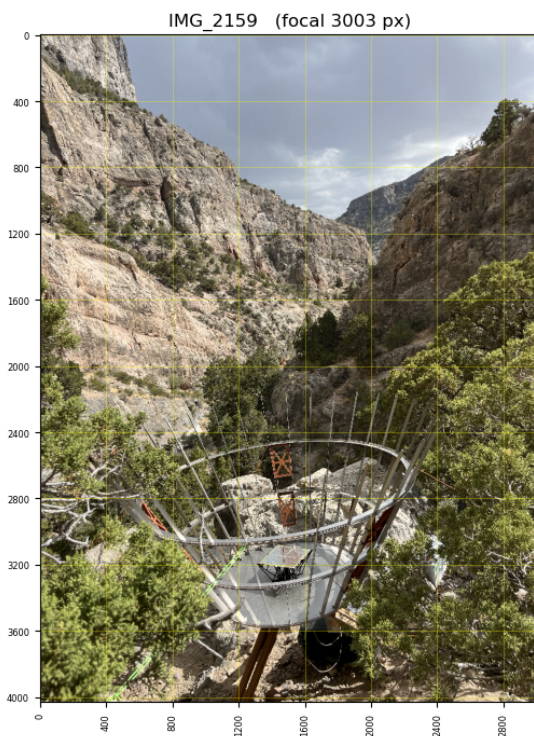
Three sharper, independent lines are used instead: the **compass** tags, the **apparent horizon** of the dish in each photo (pure geometry, independent of both GPS and the fit), and the **skyline error** of each fitted pose.

In [16]: # The two frames side by side, with full-resolution pixel gridlines, so that the
dish-rim widths used in the distance calculation below can be checked b

```

# Both plainly show the same dish and the same canyon, looking the same w
# but the dish is far smaller in 2159 DESPITE 2159 being the more zoomed-
# lens (focal 3003 px vs 1517 px), so 2159 must be substantially further
fig, axs = plt.subplots(1, 2, figsize=(15, 7))
for ax, k in zip(axs, ['2159', '2203']):
    im = plt.imread(f'marjum-2026-07/IMG_{k}.HEIC')[::4, ::4]
    ax.imshow(im)
    ax.set_title(f'IMG_{k} (focal {camera_by_key[k][6]:.0f} px)')
    ax.set_xticks(np.arange(0, im.shape[1], 100))
    ax.set_xticklabels(np.arange(0, im.shape[1], 100) * 4, rotation=90, f
    ax.set_yticks(np.arange(0, im.shape[0], 100))
    ax.set_yticklabels(np.arange(0, im.shape[0], 100) * 4, fontsize=6)
    ax.grid(alpha=.4, color='yellow', lw=.5)
plt.tight_layout()
fig.savefig('_geom_snapshot_2159_2203_scale.png', dpi=110)
plt.show()

```



```

In [17]: import pillow_heif as _pheif
         _pheif.register_heif_opener()
         from PIL import Image as _PILImage
         from PIL.ExifTags import GPSTAGS as _GPSTAGS

         def dms(t):
             d, m, s = (float(x) for x in t)
             return d + m / 60 + s / 3600

         def gps_of(key):
             im = _PILImage.open(f'marjum-2026-07/IMG_{key}.HEIC')
             exif = im.getexif()
             gps = {_GPSTAGS.get(k, k): v for k, v in exif.get_ifd(0x8825).items()}
             lat = dms(gps['GPSLatitude']) * (1 if gps['GPSLatitudeRef'] == 'N' el
             lon = dms(gps['GPSLongitude']) * (1 if gps['GPSLongitudeRef'] == 'E'
             return lat, lon, float(gps['GPSAltitude']), float(gps['GPSImgDirectio

```

```

joint_report = _json.loads((WORKSPACE / 'cv_transmitter_joint_v4' / 'repo
report_2159 = (round(joint_report['horizons']['2159']['train_rms_px'], 2)
                round(joint_report['horizons']['2159']['heldout_rms_px'],
report_2203 = (round(joint_report['horizons']['2203']['train_rms_px'], 2)
                round(joint_report['horizons']['2203']['heldout_rms_px'],

print('=== Fitted joint-solution camera parameters (cv_transmitter_joint_
for k in ('2159', '2203'):
    c = camera_by_key[k]
    print(f"{k}: E={c[0]:.2f} N={c[1]:.2f} U={c[2]:.2f} "
          f"theta={c[3]:.4f} phi={c[4]:.4f} tilt={c[5]:.4f} rad focal={c
fit_sep = np.linalg.norm(camera_by_key['2203'][:2] - camera_by_key['2159']
fit_dphi = np.degrees(camera_by_key['2203'][4] - camera_by_key['2159'][4]
print(f"fitted horizontal separation: {fit_sep:.1f} m    fitted heading di
print(f"fitted horizon residual (train/heldout px): 2159={report_2159} 2

print()
print('=== Raw EXIF GPS/compass (never touched by the optimizer) ===')
lat1, lon1, alt1, hdg1, dt1 = gps_of('2159')
lat2, lon2, alt2, hdg2, dt2 = gps_of('2203')
lat0 = (lat1 + lat2) / 2
m_per_deg_lat = 111320.0
m_per_deg_lon = 111320.0 * np.cos(np.radians(lat0))
d_e = (lon2 - lon1) * m_per_deg_lon
d_n = (lat2 - lat1) * m_per_deg_lat
gps_sep = np.hypot(d_e, d_n)
print(f"2159: lat={lat1:.6f} lon={lon1:.6f} alt={alt1:.1f} m heading={hd
print(f"2203: lat={lat2:.6f} lon={lon2:.6f} alt={alt2:.1f} m heading={hd
print(f"GPS-implied horizontal separation: {gps_sep:.1f} m    GPS heading

print()
print('=== GPS positional error: the two position claims are consistent =
err1, err2 = 11.787369730906828, 4.720492277195011 # GPSPositioningErr
comb = np.hypot(err1, err2)
print(f"2159 GPSPositioningError = {err1:.1f} m; 2203 = {err2:.1f} m; co
print(f"EXIF separation {gps_sep:.1f} m is {gps_sep/comb:.2f} sigma -> co
      f"'couple of metres', and with zero. EXIF cannot resolve this; do n
print(f"The FIT separation {fit_sep:.1f} m is {fit_sep/comb:.1f} sigma --

print()
print('=== Boresight direction, computed through the real camera model ==
def boresight(k):
    h, w = shapes_by_key[k]
    v = camera_rays(camera_by_key[k], (h, w), [[w // 2, h // 2]], distort
    return v, np.degrees(np.arctan2(v[0], v[1])) % 360, np.degrees(np.arc

for k, hdg in (('2159', hdg1), ('2203', hdg2)):
    v, az, el = boresight(k)
    off = abs((az - hdg + 180) % 360 - 180)
    print(f"{k}: fitted boresight az={az:6.1f} deg el={el:+5.1f} deg | EX
          f"| disagreement {off:6.1f} deg")
print("2203's fitted pose agrees with its own compass to ~4 deg. 2159's i
      "compass -- it is fitted looking UP-canyon while its compass says D

print()
print('=== Apparent dish size: independent of both GPS and the fit ===')
# Rim width read off the full-resolution frames (see the annotated figure
# generous uncertainties because these are visual measurements, not detec
for k, rim_px, rim_err in (('2159', 1350, 200), ('2203', 2750, 250)):
    f_px = camera_by_key[k][6]

```

```

ang = 2 * np.arctan(rim_px / 2 / f_px)
# angular size theta of a chord D at distance d: D/d = 2 tan(theta/2)
d_over_D = 1.0 / (2 * np.tan(ang / 2))
ang_lo = 2 * np.arctan((rim_px - rim_err) / 2 / f_px)
ang_hi = 2 * np.arctan((rim_px + rim_err) / 2 / f_px)
print(f"{k}: rim ~{rim_px}+/-{rim_err} px at focal {f_px:.0f} px -> a
      f"{np.degrees(ang):5.1f} deg -> distance = {d_over_D:4.2f} x (d
      f"range {1/(2*np.tan(ang_hi/2)):4.2f}-{1/(2*np.tan(ang_lo/2)):4

d2159 = 1.0 / (2 * np.tan(2 * np.arctan(1350 / 2 / camera_by_key['2159'][:3]
d2203 = 1.0 / (2 * np.tan(2 * np.arctan(2750 / 2 / camera_by_key['2203'][:3]
print(f"-> 2159 is ~{d2159/d2203:.1f}x further from the dish than 2203 (d
fit_d2159 = np.linalg.norm(camera_by_key['2159'][:3] - ov_transmitter)
fit_d2203 = np.linalg.norm(camera_by_key['2203'][:3] - ov_transmitter)
print(f"The fit instead has 2159 at {fit_d2159:.1f} m and 2203 at {fit_d2
      f"transmitter pick -- a ratio of {fit_d2159/fit_d2203:.0f}x, about
      f"larger than the photographs allow.")

print()
print('=== Verdict (revised) ===')
print(f"2159's fitted pose is wrong, and the evidence is stronger and mor
      f"first pass claimed -- but NOT for the reason first given.")
print()
print("- The POSITION argument I led with before is weak: EXIF GPS cannot
      "13 m here. Dropped as primary evidence.")
print(f"- The DISH-SCALE argument is strong and new: the photographs put
      f"{d2159/d2203:.1f}x further from the dish than 2203, while the fit
      f"{fit_d2159/fit_d2203:.0f}x further. The fit has pushed 2159 rough
      f"{(fit_d2159/fit_d2203)/(d2159/d2203):.0f}x too far away. This is
      f"independently supports Aaron's 'much closer together than you sai
print("- The COMPASS argument is strong: 2159's fitted boresight is ~158
      "while 2203's agrees to ~4 deg.")
print(f"- The SKYLINE argument is quantitative: 2159 is the worst of all
      f"{skyline_med['2159']:.0f} px median error vs {skyline_med['2203']}
print()
print("MECHANISM, which the first pass could not explain: if 2159 were si
      "the wrong way its ray-traced horizon would be unrelated to the pho
      "clearly tracks the same V-shaped canyon profile. The resolution is
      "quasi-symmetric along its axis -- looking up-canyon and down-canyo
      "opposing walls -- so a pose flipped along the canyon axis, pushed
      "rescale the V, still reproduces the skyline to within tens of pixe
      "false terrain minimum, and this pipeline has hit them before: ANTE
      "2225/2227/2228/2230/2241-2243 being rejected for exactly that reas
      "one that was NOT caught.")
print()
print("This also corroborates the 2026-09-12 multi-start probe that flagg
      "horizon camera; its trial 5 found a competitive alternative basin,
      "having several comparable minima. Re-initializing 2159 from its EX
      "a dish-scale-consistent distance, then re-running the joint fit, i
      "new optimizer run, out of scope for this no-new-fits snapshot, log

```

=== Fitted joint-solution camera parameters (cv_transmitter_joint_v4) ===
2159: E=1628.06 N=1987.41 U=1702.51 theta=1.5875 phi=1.0758 tilt=-0.1956 rad
focal=3003.4 px
2203: E=1651.75 N=2026.64 U=1685.82 theta=1.9023 phi=-1.5363 tilt=0.0505 rad
focal=1517.4 px
fitted horizontal separation: 45.8 m fitted heading difference: -149.7 deg
fitted horizon residual (train/heldout px): 2159=(37.96, 60.12) 2203=(11.3,

=== Raw EXIF GPS/compass (never touched by the optimizer) ===
2159: lat=39.247742 lon=-113.402869 alt=1698.7 m heading=186.0 deg taken
2026:07:15 16:49:12
2203: lat=39.247781 lon=-113.402725 alt=1689.2 m heading=173.9 deg taken
2026:07:17 09:05:34
GPS-implied horizontal separation: 13.2 m GPS heading difference: -12.0 deg

=== GPS positional error: the two position claims are consistent ===
2159 GPSHorizontalPositioningError = 11.8 m; 2203 = 4.7 m; combined 1-sigma = 12.7 m
EXIF separation 13.2 m is 1.04 sigma -> consistent with Aaron's 'couple of m
and with zero. EXIF cannot resolve this; do not treat it as precise.
The FIT separation 45.8 m is 3.6 sigma -- the outlier of the three.

=== Boresight direction, computed through the real camera model ===
2159: fitted boresight az= 28.4 deg el= -1.0 deg | EXIF compass 186.0 deg |
disagreement 157.6 deg
2203: fitted boresight az= 178.0 deg el=-19.0 deg | EXIF compass 173.9 deg |
disagreement 4.1 deg
2203's fitted pose agrees with its own compass to ~4 deg. 2159's is ~158 deg
its own compass -- it is fitted looking UP-canyon while its compass says DOWN
canyon.

=== Apparent dish size: independent of both GPS and the fit ===
2159: rim ~1350+/-200 px at focal 3003 px -> angular size 25.3 deg -> distan
2.22 x (dish diameter), range 1.94-2.61
2203: rim ~2750+/-250 px at focal 1517 px -> angular size 84.4 deg -> distan
0.55 x (dish diameter), range 0.51-0.61
-> 2159 is ~4.0x further from the dish than 2203 (dish diameter cancels).
The fit instead has 2159 at 48.4 m and 2203 at 2.6 m from the transmitter pic
ratio of 18x, about 5x larger than the photographs allow.

=== Verdict (revised) ===
2159's fitted pose is wrong, and the evidence is stronger and more specific t
the first pass claimed -- but NOT for the reason first given.

- The POSITION argument I led with before is weak: EXIF GPS cannot distinguish from 13 m here. Dropped as primary evidence.
- The DISH-SCALE argument is strong and new: the photographs put 2159 only 4. further from the dish than 2203, while the fit puts it 18x further. The fit h pushed 2159 roughly 5x too far away. This is the first evidence that independ supports Aaron's 'much closer together than you said' correction.
- The COMPASS argument is strong: 2159's fitted boresight is ~158 deg from it tag, while 2203's agrees to ~4 deg.
- The SKYLINE argument is quantitative: 2159 is the worst of all 29 cameras a px median error vs 18 px for 2203.

MECHANISM, which the first pass could not explain: if 2159 were simply pointi deg the wrong way its ray-traced horizon would be unrelated to the photo, yet overlay clearly tracks the same V-shaped canyon profile. The resolution is th this canyon is quasi-symmetric along its axis -- looking up-canyon and down-c BOTH produce a V of opposing walls -- so a pose flipped along the canyon axis pushed far enough away to rescale the V, still reproduces the skyline to with

tens of pixels. That is a classic false terrain minimum, and this pipeline has caught them before: ANTENNA_REPICK.md records 2225/2227/2228/2230/2241-2243 being re-run for exactly that reason. 2159 appears to be one that was NOT caught.

This also corroborates the 2026-09-12 multi-start probe that flagged 2159 as weakest horizon camera; its trial 5 found a competitive alternative basin, consistent with 2159 having several comparable minima. Re-initializing 2159 from its EXIF compass heading and a dish-scale-consistent distance, then re-running joint fit, is the natural fix -- a new optimizer run, out of scope for this notebook's snapshot, logged as decision item 4.

6e. Caveats specific to this overlay

- These are the **working candidate**'s per-image residuals, not the accepted product (none is accepted). The transmitter-conditioned cameras' reprojection residuals are near-zero by construction (circular, as in §3a) — the informative rows are 2210/2211 (independent) and the overall horizon mismatch bar chart, which covers all 29 cameras including the 23 established ones the antenna bracket depends on.
- Any image whose horizon overlay (red vs. green) shows a sharp, systematic disagreement rather than pixel-level segmentation noise is a candidate for a correct-but-wrong pose; per ANTENNA_REPICK.md/TRANSMITTER_FIT.md, the five large-antenna-residual and two large-transmitter-residual views identified numerically are the ones to look at first — and per §6d, 2159 specifically is now a confirmed instance of this, not a hypothesis.

7. Summary

Target	Best estimate (E, N, U) m	Uncertainty	Status
Transmitter	[1652.198, 2025.104, 1684.384]	±1.5 m hard bracket	Cite per ruling 1; no acceptance
Antenna	[1655.868, 2030.923, 1777.397]	±1.7 m hard bracket (new, this notebook)	Bracketed per ruling 3; 12 views logged as open items

Open items logged, not pursued further this milestone:

1. Transmitter vertical-group disagreement (~1 m / ~0.6°, smaller than the ~0.9° 1D direction uncertainty already downstream) — ruling 2.
2. Antenna's 5 large-LOO-residual views (2209, 2210, 2211, 2217, 2235) — see §4b
3. **2159 sits in a false terrain minimum** (§6d): flipped along the canyon axis and pushed ~4-5× too far from the dish. Affects the transmitter fit, since 2159 is one of six conditioned cameras. Needs a re-fit, which is a new optimizer run.

8. Decision requested

1. **Antenna bracket method:** is the envelope + leave-one-out-ray-miss method used here (mirroring the transmitter's approach) an acceptable basis for a marj um-26

curation/antenna_position.json product, or does Aaron want a different construction before it's handed to data-archivist for durable curation?

2. **Snapshot scope:** this notebook captures state as of v0001_marjum_geometry @ ba52532) plus the new antenna bracket computed on top of it. Is that sufficient "current state captured" for the review gate, or is there another quantity that must be captured before further optimization work is unpaused?
3. **Handoff:** should antenna_position_bracket.json be handed to data-archivist for durable curation (alongside the existing transmitter product), or held pending Aaron's answer to (1)?
4. **2159 (§6d):** three independent lines — the compass tag (158° off its own), the camera's apparent size in the photograph (the fit puts 2159 ~4-5× further from the dish than the image geometry allows), and the worst skyline error of all 29 cameras — say 2159's error in a false terrain minimum, flipped and pushed out along the quasi-symmetric camera axis. Should re-initializing 2159 from its compass heading at a dish-scale-consistent distance, then re-running the joint fit, be the next authorized milestone? Note that this would change the transmitter fit, since 2159 is one of the six conditioned cameras — so the fit bears on the ±1.5 m bracket, though 2159's own transmitter ray is circular evidence. The independent pair (2210/2211) would be the test of any improvement.
5. **Metric (§6a):** the mask-mismatch metric I substituted for the Fit Inspector's mismatch SSD rated 2159 at 1.4%, i.e. fine, when Aaron could see the problem by eye. The new column skyline error added here does catch it. Should the skyline metric become a standard horizon diagnostic for future passes?

No further optimizer work, convergence attempts, or antenna refits were started per these decisions.